Learning to explore with partial observations

MiniGrid exploration agent | Technical report | Python 3.11

FULL EXPERIMENT MATRIX

1. Problem and information boundary

We model exploration as a partially observed decision process. The agent must reveal traversable terrain, discover points of interest (POIs), and survive a moving patrol within an action budget. The environment is an extension of Farama's MiniGrid, rather than a replacement simulator. Its native grid, visibility, turn/forward transitions, and renderer remain the foundation.

The policy receives a wall-occluded 7 x 7 symbolic view, heading, movement/contact feedback, remaining time, and a map reconstructed from those signals. The mapping module has no environment reference. Absolute map coordinates, hidden POIs, hidden patrol positions, and ground-truth coverage denominators never enter policy inputs or rewards.

A fixed 55 x 55 memory canvas is centered at the initial position. Known walls/floor, discovered POIs, clipped occupancy counts, the agent marker, and currently visible monsters form seven channels. The current view is five one-hot channels; eight scalars encode heading and feedback. Monster occupancy is cleared every observation, avoiding fictitious current positions from stale sightings.

2. Environment design

Preset	Size	POIs	Action budget	Monster period
Easy	15 x 15	2	512	3 actions
Medium	21 x 21	3	1,024	2 actions
Hard	27 x 27	5	1,536	2 actions

A seeded depth-first maze supplies corridors and dead ends. Rectangular rooms and sparse additional connections add open areas, junctions, and cycles. Connectivity and reachable placements are checked before simulation. One red monster starts at graph distance at least six, walks orthogonally, avoids immediate reversal when alternatives exist, and respects walls. Agent motion precedes collision checking, then the scheduled monster move and a second check. Contact is fatal; the ordinary budget limit is a truncation. Map and patrol RNG streams are independent.

3. Rewards and policy

The four actions are left, right, forward, and wait. The reward is computed from observation history, with each newly revealed floor cell, visited cell, and POI counted once per episode. Initial observations initialize history but pay no reward.

Component	Weight	Observable event
New floor revealed	+0.02	Previously unknown traversable tile seen
New cell visited	+0.01	First occupancy inferred through odometry
New POI discovered	+1.00	First visible green floor marker
Every action	-0.001	Action taken
Wall contact	-0.02	Contact feedback
Monster contact	-1.00	Fatal contact feedback

No reward clipping or running reward normalization is used. Turning can legitimately reveal a new area once; repeated turns, revisits, and re-sightings cannot repeatedly earn discovery reward. The time cost discourages camping. Remaining risks include prioritizing an immediately rewarding discovery over survival and learning short cycles; therefore outcomes and loop metrics are reported independently of training return.

Stable-Baselines3 PPO uses separate encoders for the local view (16/32-channel convolutions, then 64 features) and reconstructed map (32/64/64 stride-two convolutions, then 128 features). Their outputs plus eight scalars feed separate two-layer 128-unit actor and critic heads. This approach makes memory auditable and avoids adding recurrent-state management to a short engineering assignment. It assumes exact movement feedback and a known maximum map size.

Default training: 16 environments; 256 steps per rollout per environment; minibatches of 256; four epochs; learning rate 0.0003; gamma 0.99; GAE lambda 0.95; clipping 0.2; entropy coefficient 0.01; target KL 0.03; value coefficient 0.5; gradient norm bound 0.5. Training samples an equal-probability mixture of Easy and Medium maps. Advantage normalization is PPO's default. KL, entropy, losses, coverage, discoveries, survival, and loops are logged.

The planned matrix trains main, no-memory, and no-novelty variants for seeds 11, 22, and 33, each to five million transitions. PPO rounds budgets to complete rollouts. The no-memory variant zeros the policy's map branch while retaining internal discovery accounting; no-novelty removes the floor-reveal and first-visit bonuses. Everything else is held fixed.

Packaged checkpoint: variant **main**, training seed **11**, selected step **4,250,000**. Runtime dependencies: MiniGrid 3.2.0; SB3 2.9.2; PyTorch 2.11.0+cu128.

Recorded training host: NVIDIA GeForce RTX 4090 (24.0 GiB), 20 logical CPU cores, NVIDIA driver 610.88. PyTorch uses one CPU thread per training/inference process; policy evaluation uses CPU. See results/hardware.json and per-run runtime.json for provenance and measured throughput.

4. Evaluation methodology

Map-seed partitions are training 0-9,999, validation 10,000-10,049, and held-out test 20,000-20,099. Training RNG seeds are distinct from map seeds. A fixed seed and action sequence reproduces the trajectory. Checkpoints are selected on validation mean coverage AUC, with POI discovery and then survival as lexicographic tie-breakers. Test outcomes do not select checkpoints.

The random baseline uniformly samples all four actions, with policy seeds 11, 22, and 33. The deterministic frontier baseline uses BFS only through observed floor, turns to inspect frontier cells, uses fixed tie-breaking, and treats visible monsters and adjacent cells as unsafe. Both baselines use the same seeds, budgets, and information restrictions as PPO. The frontier agent is an intentionally strong reference, not an oracle.

Metric	Definition
Revealed / visited coverage	Unique observed / occupied floor divided by true reachable floor
Coverage AUC	Trapezoidal coverage over the full action budget, normalized by budget
POI discovery	Unique observed POIs divided by total generated POIs
Survival / completion	Reaches the configured action limit without fatal contact
Movement efficiency	New occupied cells after reset divided by actions
Repeated-state / loop rate	Recent (position, heading) repeat / repeat with no discovery, window 16
Threat / wall interactions	Fatal contacts and wall-contact count/rate
Inference latency	Action selection plus observation/memory preprocessing; engine excluded

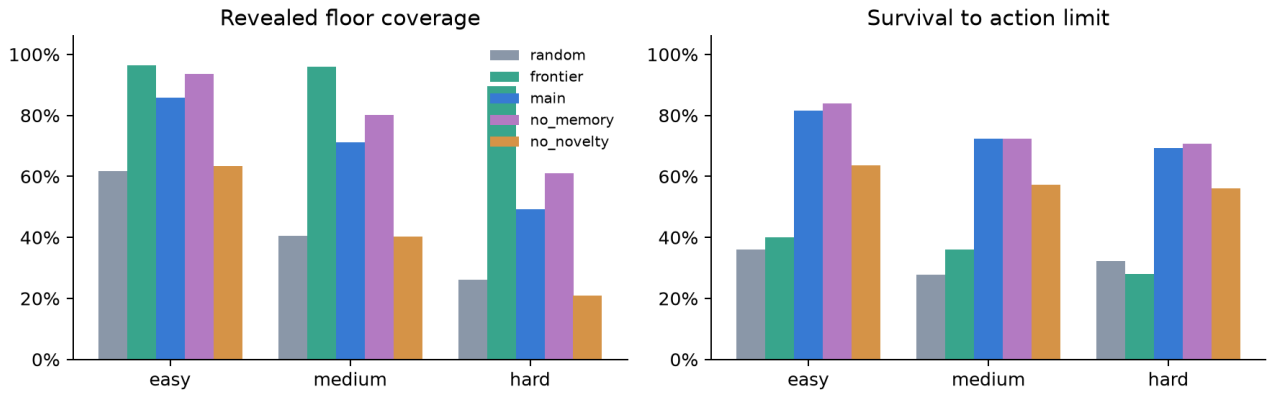
The evaluator alone knows the true terrain and total POI count. After death, final coverage is held constant through the remaining budget when computing AUC and curves, so early deaths cannot improve averages by excluding later timesteps. Latency includes the first action and is reported as mean, median, and p95. CSVs preserve per-episode results and termination reasons.

Episode standard deviation and standard deviation of training-run means are separate. Paired bootstrap intervals average runs per map and resample shared maps; they quantify map variation, not full training uncertainty. Hard maps test extrapolation to a size absent from training. Training-partition diagnostic evaluation, when exported, uses the same evaluation command with --split train; no train/test gap is inferred from training returns.

This report's actual evaluation: **test**; 100 map seeds per preset (20000-20099); 15 agent/preset groups. Raw sources: episodes.csv, curves.csv, summary.json in the evaluation output directory.

5. Measured results

Agent / preset	Episodes	Coverage %	AUC	POI	Survival
frontier/easy	100	96.5 +/- 10.0	0.931	96.5%	40.0%
frontier/hard	100	89.6 +/- 20.0	0.833	90.0%	28.0%
frontier/medium	100	96.0 +/- 14.1	0.912	95.3%	36.0%
main/easy	300	85.8 +/- 21.6	0.797	86.2%	81.7%
main/hard	300	49.1 +/- 24.7	0.437	46.5%	69.3%
main/medium	300	71.2 +/- 24.7	0.635	70.0%	72.3%
no_memory/easy	300	93.5 +/- 12.5	0.863	92.3%	84.0%
no_memory/hard	300	61.1 +/- 23.0	0.516	60.3%	70.7%
no_memory/medium	300	80.1 +/- 20.0	0.705	80.0%	72.3%
no_novelty/easy	300	63.4 +/- 27.3	0.575	61.8%	63.7%
no_novelty/hard	300	21.0 +/- 12.8	0.191	19.1%	56.0%
no_novelty/medium	300	40.2 +/- 21.0	0.361	40.1%	57.3%
random/easy	300	61.9 +/- 21.7	0.558	59.0%	36.0%
random/hard	300	26.2 +/- 12.9	0.228	25.3%	32.3%
random/medium	300	40.5 +/- 18.3	0.356	38.4%	27.7%



Coverage is mean +/- episode standard deviation in percentage points. Bars show means. Run means, run-mean variation, wall contacts, loops, latency, and paired comparison intervals are in summary.json. No claim that PPO beats the frontier baseline is assumed.

Easy: main minus random coverage AUC = +0.239; paired-map 95% interval [+0.207, +0.273] across 100 maps.

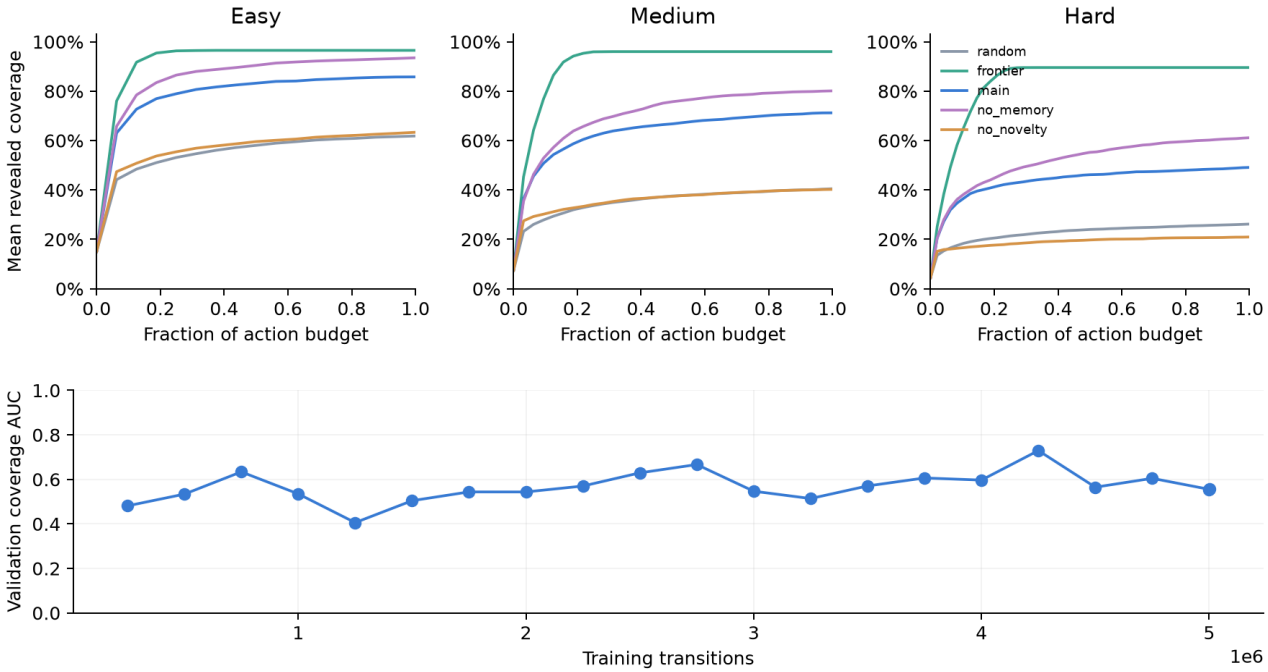
Hard: main minus random coverage AUC = +0.209; paired-map 95% interval [+0.181, +0.239] across 100 maps.

Medium: main minus random coverage AUC = +0.279; paired-map 95% interval [+0.245, +0.314] across 100 maps.

Ablation: main minus no_memory mean coverage AUC across training-sized presets = -0.068. Run variation is retained in the raw summary.

Ablation: main minus no_novelty mean coverage AUC across training-sized presets = +0.247. Run variation is retained in the raw summary.

6. Behavior, limitations, and reproducibility



Failure inspection: main seed 11, hard map 20055, reached 1.3% coverage with 99.9% looping and termination 'time_limit'. Full action traces, evaluator/memory panels, final frames, and separately ranked low-coverage, high-loop, and fatal cases support inspection. Replay metadata records skipped video frames and every action.

Known limitations: perfect compass/contact odometry simplifies localization; only one procedural generator family and one patrol rule are studied; frontier avoidance is reactive; mortality makes exploration/survival a genuine trade-off; deterministic PPO inference may expose cycles that stochastic training hides. The fixed map canvas supports sizes through 27 only. CPU latency is machine-dependent. Checkpoint resume restarts environment sampling and is not bitwise continuation. Different map seeds do not guarantee different geometry, although generator diversity is tested.

The interactive Tkinter inspector exposes seeds, presets, size, POIs, horizon, patrol interval, agent/checkpoint selection, playback speed, and start/pause/reset/single-step controls. Its evaluator view is clearly labeled. A PyInstaller Windows CPU build includes the model, metadata, and Python runtime; smoke mode runs without a GPU or retraining.

Reproduce: install the locked CPU or CUDA dependencies, run pytest tests_exploration, train with configs/main.yaml, evaluate saved checkpoints using the committed seed partition, then export this report and demo. scripts/run_matrix.py resumes interrupted runs and evaluates the held-out partition only after every requested training run completes. Logs, configs, checkpoint checksums, runtime metadata, and raw episode metrics are retained.

Generalization: main test minus training-partition coverage AUC averages +0.002 on Easy/Medium; Hard minus Medium test AUC is -0.198. Exact coverage, POI, and survival gaps are in generalization.json. Training-partition samples are not guaranteed to have been encountered by every run.

Future work: uncertain odometry, beliefs about hidden threats, recurrent-policy comparisons, richer map families, and more independent training seeds. These are extensions, not evidence supplied here.

References and attribution: Farama Foundation MiniGrid (MIT), github.com/Farama-Foundation/Minigrid; Stable-Baselines3 (MIT), stable-baselines3.readthedocs.io; PyTorch (BSD-style); Gymnasium (MIT). See ATTRIBUTION.md and bundled third-party license files. PPO: Schulman et al., Proximal Policy Optimization Algorithms (2017), arXiv:1707.06347.